



SEC 3213: Software Security

2nd Trimester: 2024
Software Security Course Project
Developing a Secure Web Application

Name	ID	Group number
Elaf Alsaidi	444003913	1
Razan Almalki	444003435	1
Ghadah Alharthi	444006163	1
Wafaa Alawadhi	443011955	1
Retaj Baaqeel	444006589	1

Table of Contents

Phase Two: Threat Modelling and Risk Assessment	3
Know your system!	3
Beauty & Glow Data Flow Diagram	3
Description of the DFD	3
Components of the DFD	3
Identify Potential Threat Using STRIDE Methodology	5
STRIDE Table	5
Risk Assessment and Prioritization	7
Risk Matrix Table	7
Risk Matrix Analysis Table	7
Mitigation	9
Mitigation Table	9
Phase Three: Secure Development Practices.....	10
Security Functions Table	10
Phase Four: Security Analysis and Testing	14
HTML Analysis	14
CSS Analysis	15
PHP Analysis	17
JavaScript Analysis	19

Phase Two: Threat Modeling and Risk Assessment

Know your system!

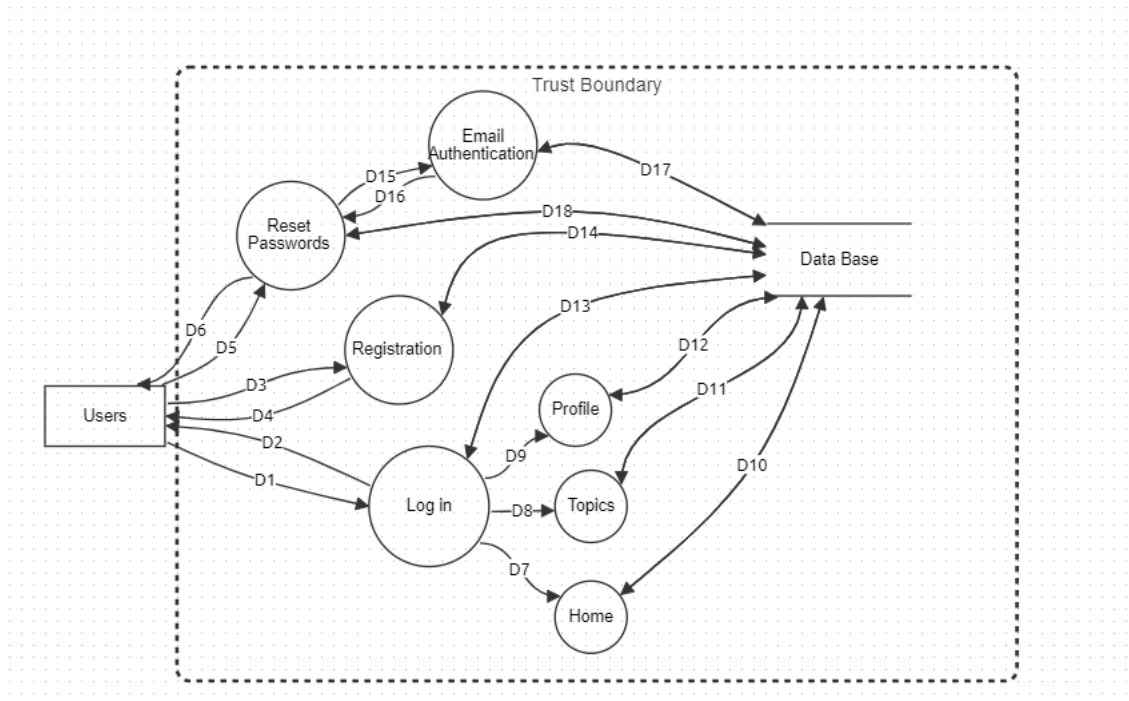


Figure 1 Beauty & Glow Data Flow Diagram

Description of the Diagram:

Beauty & Glow DFD displays the interaction between the users and the three pages (processes) which are Reset Passwords, Registration and Log in pages in the beginning, because the users are restricted to viewing only these three pages until they log in to our website. After successful login, the rest of the pages will be available. All processes have access to the data base to store the information and to take it from the data base. Finally, the Email authentication process provides authentication functionality for users who forget the password or want to reset the password.

Components of the DFD:

External Entity:

- **Users (D1):** This box represents the external users interacting with the system. They initiate processes such as registration and login.

Processes:

- **Registration (D2):** This process handles new user registrations. It collects user information, which is stored in the database.
- **Log In (D3):** This process allows users to access their accounts by verifying their credentials against the stored data.
- **Reset Passwords (D4):** This process enables users to change their passwords.
- **Email Authentication (D5):** This process involves sending a verification email to users for confirming their email addresses.

- **Profile (D6):** This process allows users to view and edit their personal information.
- **Topics (D7):** This process involve uploading topics relevant to the user.
- **Home (D8):** This is the main dashboard or homepage users see after logging in and provides the user's topics and managing the topics.

Data Stores:

- **Database (D9):** The central data store where user information, profiles, topics, and other relevant data are stored.

Data Flows:

- **D1 (Users to Log in):** Users enter their credentials to Login, which are username and password.
- **D2 (Log in to Users):** The response for the user login request.
- **D3 (Users to Registration):** Users enter their information to Registration, which are username, email and password.
- **D4 (Registration to Users):** The response for the user Registration request. confirming whether registration was successful or not.
- **D5 (Users to Reset Passwords):** Users request to reset their passwords, typically providing their email address.
- **D15(Reset Passwords to Email Authentication):** After the reset request send, the system generates a link to the reset page and sends it via an email to enable reset the password for the user and to ensure authentication.
- **D16, D6 (Email Authentication to Users):** Users receive an email containing the password reset link.
- **D7 (Users to Home):** Users navigate to the home page after successfully logging in.
- **D8(Users to Topics):** Users upload information about specific topics they are interested in.
- **D9 (Users to Profile):** Users access their profile to view or edit personal information.
- **D10, D11, D12(Home, Topics and Profile to Database and vice versa):** The information related to these pages are stored in the database and any information management performed will be updated in the database.
- **D13 (Log in to Database and vice versa):** The system checks the provided credentials against the stored data in the database
- **D14 (Registration to Database And vice versa):** Registered user information is stored in the database. Registered user information is taken from database.
- **D17 (Email Authentication to Database and vice versa):** The system logs the email authentication process, ensuring that the user's email is verified before allowing a password reset.
- **D18 (Reset Passwords to Database and vice versa):** The system updates the user's password in the database after successful authentication.

Trust Boundary:

- The dotted line around the processes indicates a trust boundary, separating external entities (Users) from the internal processes and data stores. This boundary highlights the areas where security measures should be implemented to protect sensitive data.

Identify Potential Threat Using STRIDE Methodology

Threat Number	Threat Type	The Threatened component	Description
1	Spoofing	D1 (Log In)	Attack Name: (Credential Theft). An attacker impersonates a legitimate user to gain unauthorized access to the system using stolen credentials.
2	Tampering	D14 (Registration)	Attack Name: (SQL Injection). An attacker alters user data during registration by injecting malicious SQL queries, potentially compromising user information.
3	Repudiation	D1 (Log In), D5 (Reset Passwords)	Attack Name: (Action Denial). A user denies having performed an action (e.g., logging in or changing a password), complicating accountability and auditing.
4	Information Disclosure	Database	Attack Name: (Unauthorized access). Unauthorized users may gain access to sensitive data stored in the database due to weak access controls or unencrypted data.
5	Denial of Service	D1 (Users to Log In), D3 (Users to Registration)	Attack Name: (DDoS Attack or brute force attacks). An attacker overwhelms the system with requests, making it unavailable for legitimate users trying to access login or registration services.
6	Elevation of Privilege	D9 (Profile), D8 (Topics)	Attack Name: (Privilege Escalation). An attacker exploits a vulnerability to gain higher privileges than intended, allowing unauthorized access to sensitive areas.
7	Spoofing	D15 (Reset Passwords to Email Authentication)	Attack Name: (Email Spoofing). An attacker sends a fake password reset email that appears legitimate, tricking users into clicking malicious links.
8	Tampering	D7 (Home)	Attack Name: (session hijacking). an attacker intercepts and modifies the session between a client and a server. usually through Cross-site scripting (XSS) Exploiting vulnerabilities in a website to steal session cookies.
9	Repudiation	D11 (topics to database)	Attack Name: (Action Denial). Not able to verify who upload a topic in our website
10	Information Disclosure	D1 (Users to Log In)	Attack Name: (Man-in-the-Middle (MitM)). An attacker intercepts the

			communication between the user and the server to capture login credentials.
11	Denial of Service	D8 (Users to Topics)	Attack Name: (Resource Exhaustion) An attacker attempts to upload excessive topics to exhaust server resources, impacting performance for legitimate users.
12	Elevation of Privilege	D3 (Users to Registration)	Attack Name: (Privilege Escalation). An attacker registers using a specially crafted email address that exploits validation flaws, granting them elevated rights.

Table 2 Potential Threat (STRIDE)

Risk Assessment and Prioritization

Risk Matrix		Consequences				
		Negligible (1) Minimal injuries / no injuries	Minor (2) Minor injuries / first aid	Significant (3) Moderate injuries / medical treatment	Severe (4) Serious injuries / hospitalisation	Catastrophic (5) Death / permanent impairment
Likelihood	Certain (5) 100% likely / almost 100% likely	Moderate (5)	High (10)	High (15)	Catastrophic (20)	Catastrophic (25)
	Likely (4) Will probably happen / is likely to happen	Moderate (4)	Moderate (8)	High (12)	Catastrophic (16)	Catastrophic (20)
	Possible (3) Could happen or plausible	Low (3)	Moderate (6)	Moderate (9)	High (12)	High (15)
	Unlikely (2) Improbable but could happen / not expected	Low (2)	Moderate (4)	Moderate (6)	Moderate (8)	High (10)
	Very Unlikely (1) Rare / not expected but remotely possible	Low (1)	Low (2)	Low (3)	Moderate (4)	Moderate (5)

Table 2 Risk Matrix

Threat Number	Threat Specific	Likelihood	Consequence	Risk Score Likelihood x Consequence
1	Spoofing (Credential Theft).	4	4	16
2	Tampering (SQL Injection).	4	5	20
3	Repudiation (Action Denial).	2	2	4
4	Information Disclosure (Unauthorized access).	3	5	15
5	Denial of Service (DDoS Attack or brute force attacks).	3	4	12
6	Elevation of Privilege (Privilege Escalation).	3	4	12

7	Spoofing (Email Spoofing).	4	3	12
8	Tampering (session hijacking).	4	4	16
9	Repudiation (Action Denial).	2	2	4
10	Information Disclosure (Man-in-the-Middle (MitM)).	2	4	8
11	Denial of Service (Resource Exhaustion)	3	4	12
12	Elevation of Privilege (Privilege Escalation).	3	4	12

Table 3 Risk Matrix Analysis

Mitigation

Threat Number	Threat Specific	Mitigation
1	Spoofing (Credential Theft).	account lockout mechanisms after multiple failed login attempts. Implement multi-factor authentication (MFA)
2	Tampering (SQL Injection).	Use encryption for data in transit and at rest. Employ input validation and parameterized queries to prevent SQL injection.
3	Repudiation (Action Denial).	Implement robust logging mechanisms that track user actions in a non-repudiable manner.
4	Information Disclosure (Unauthorized access).	Enforce strict access controls, use encryption for sensitive data, and conduct regular security audits.
5	Denial of Service (DDoS Attack or brute force attacks).	Implement rate limiting, use web application firewalls (WAFs), and monitor traffic for unusual patterns.
6	Elevation of Privilege (Privilege Escalation).	Regularly update software to patch vulnerabilities
7	Spoofing (Email Spoofing).	Spread awareness of the dangers of some emails and verify the validity of the email sent before entering any email or clicking on any link.
8	Tampering (session hijacking).	Set short session expiration times, requiring re-authentication after a period of inactivity.
9	Repudiation (Action Denial).	Maintain comprehensive logs of the system events And implement a backups database
10	Information Disclosure (Man-in-the-Middle (MitM)).	Implement HTTPS for all communications and use strong encryption protocols.
11	Denial of Service (Resource Exhaustion)	Implement size limits on uploads and monitor for unusual activity patterns.
12	Elevation of Privilege (Privilege Escalation).	Use validation checks Implement input validation to prevent unauthorized access and privilege escalation

Table 4 Mitigation

Phase Three: Secure Development Practices

	Security Function Used	OWASP Category	Vulnerability Prevented	Security Measure Category	Function Job Description
1	Prepared statements (mysqli_prepare(), bind_param(), execute())	A1: Injection	SQL injection attacks.	Input Validation, Data Sanitization	Prevents SQL injection by separating SQL code from user-supplied data. Data is parameterized, preventing malicious code execution.
2	htmlspecialchars() with ENT_QUOTES	A7: Cross-Site Scripting (XSS)	Cross-Site Scripting (XSS)	Output Encoding	Converts special characters (like <, >, ", ') into their HTML entities, preventing them from being interpreted as code.
3	\$resetToken = time();	A2: Broken Authentication	Replay attacks	Session Management, Password Reset	Generates a reset token using the current timestamp.
4	filter_input() with FILTER_SANITIZE_STRING	A1: Injection, A7: XSS	Cross-Site Scripting (XSS), Command Injection, SQL Injection	Input Validation, Data Sanitization	Filters user input, removing potentially harmful characters from strings.
5	filter_input() with FILTER_VALIDATE_EMAIL	A1: Injection, A7: XSS	Invalid email format, Cross-Site Scripting (XSS), SQL Injection	Input Validation, Data Sanitization	Validates user input to ensure it's a valid email address format.
6	strlen()	A1: Injection, A7: XSS	Prevents overly long inputs leading to buffer overflows or injection attacks	Input Validation	Measures the length of a string. Used in input validation to prevent overly long inputs that could cause buffer overflows.

7	<code>\$stmt->bind_param()</code>	A1: Injection	SQL Injection	Input Validation, Data Sanitization	Binds parameters to a prepared statement, preventing SQL injection.
8	<code>password_hash()</code>	A2: Broken Authentication	Weak passwords, credential stuffing	Password Management	Hashes passwords securely using a strong one-way hashing algorithm
9	<code>error_log()</code>	A10: Insufficient Logging & Monitoring	Difficulty in detecting and responding to security incidents	Logging and Monitoring	Logs errors and events to a log file for auditing and debugging.
10	<code>\$_SESSION['csrf_token'] = bin2hex(random_bytes(32));</code>	A8: Cross-Site Request Forgery (CSRF)	Cross-Site Request Forgery (CSRF)	CSRF Protection	Generates a unique, unpredictable CSRF token for each session to protect against CSRF attacks.
11	<code>(error_log())</code> Login attempts are logged	A2: Broken Authentication, A10: Insufficient Logging & Monitoring	Brute-force attacks, unauthorized login attempts	Logging and Monitoring	Logs login attempts (successful and failed) for security auditing and detection of suspicious activity.
12	<code>\$otp = rand(100000, 999999);</code>	A2: Broken Authentication	Brute-force attacks, unauthorized login attempts	Password Reset, Multi-factor Authentication	Generates a 6-digit random number as an OTP.
13	PHPMailer	A3: Sensitive Data Exposure, A2:	Email address exposure, insecure email sending	Email Security	Sends emails; security depends heavily on proper configuration (TLS/SSL, authentication, etc.).

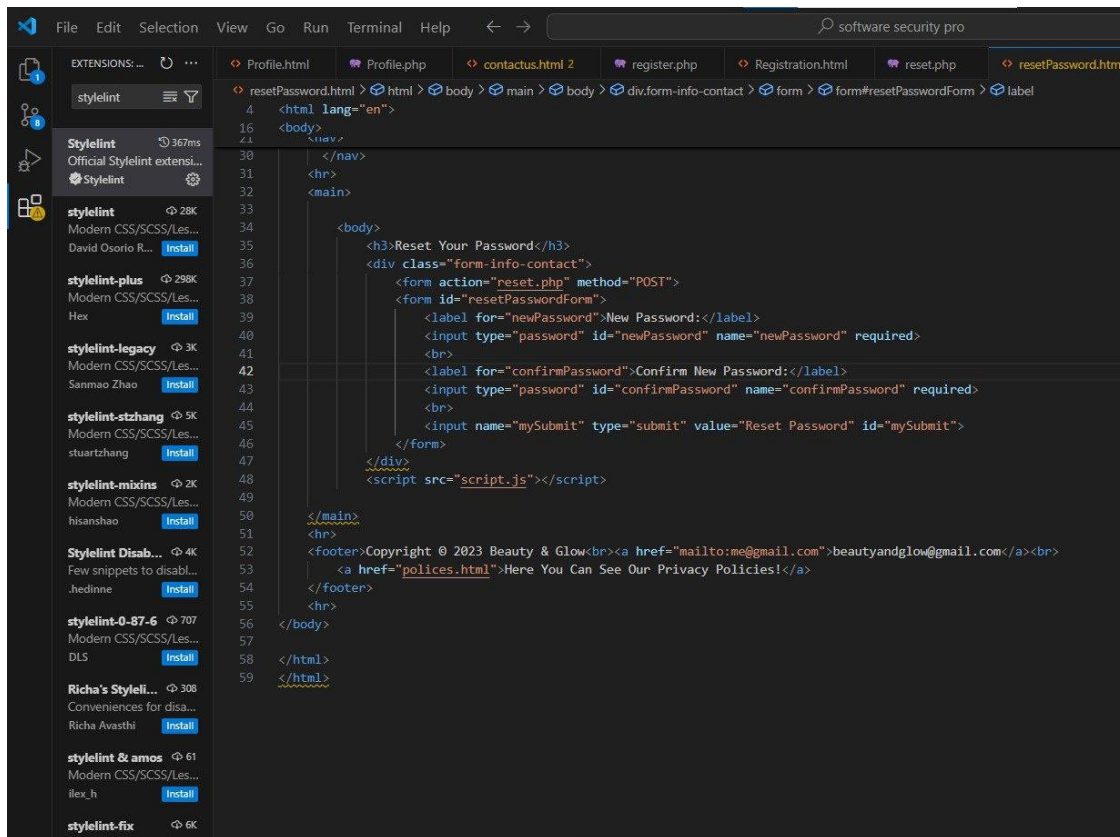
		Broken Authentication			
14	<code>\$mail->SMTPSecure = PHPMailer::ENCRYPTION_SMTPS;</code>	A3: Sensitive Data Exposure	Email transmission interception	Email Security	sets the security mode for the SMTP connection used by PHPMailer to send emails.
15	<code>\$mail->SMTPAuth = true;</code>	A3: Sensitive Data Exposure	Unauthorized email sending	Email Security	enables SMTP authentication for the PHPMailer email sending process.
16	<code>session_start()</code>	A2: Broken Authentication	Session hijacking, unauthorized access	Session Management	Initializes a PHP session, allowing the application to store and retrieve user data across multiple requests.
17	<code>!isset(\$_SESSION['username'])</code>	A2: Broken Authentication	Unauthorized access	Session Management	Checks if a user session exists. If not, it redirects the user to the login page, preventing unauthorized access to the profile page.
17	<code>(time() - \$_SESSION['login_time'] > 1800)</code>	A2: Broken Authentication	Session timeout, prolonged unauthorized access	Session Management	Checks if the session has expired (30 minutes of inactivity). If it has, the session is destroyed, and the user is redirected to the login page.
18	<code>session_unset(), session_destroy()</code>	A2: Broken Authentication	Prevents unauthorized access after session expiry	Session Management	Clears and destroys the session data, ensuring
19	<code>move_uploaded_file()</code>	A3: Sensitive Data Exposure, A6: Security Misconfiguration	File upload vulnerabilities (arbitrary file upload)	File Upload Handling	Moves the uploaded file to a specified directory.

20	password_verify(\$password, \$row['password'])	A2: Broken Authenti cation	password storage	Passwor d Manage ment	Verifies the user's password against the securely hashed password stored in the database. This is a crucial step for secure authentication
21	\$stmt->bind_param("ss", \$username, \$topic)	A1: Injection	SQL Injection	Paramete rized Queries	Binds parameters to a prepared statement, preventing SQL injection by separating data from the SQL query.

Table 5 security functions

Phase Four: Security Analysis and Testing

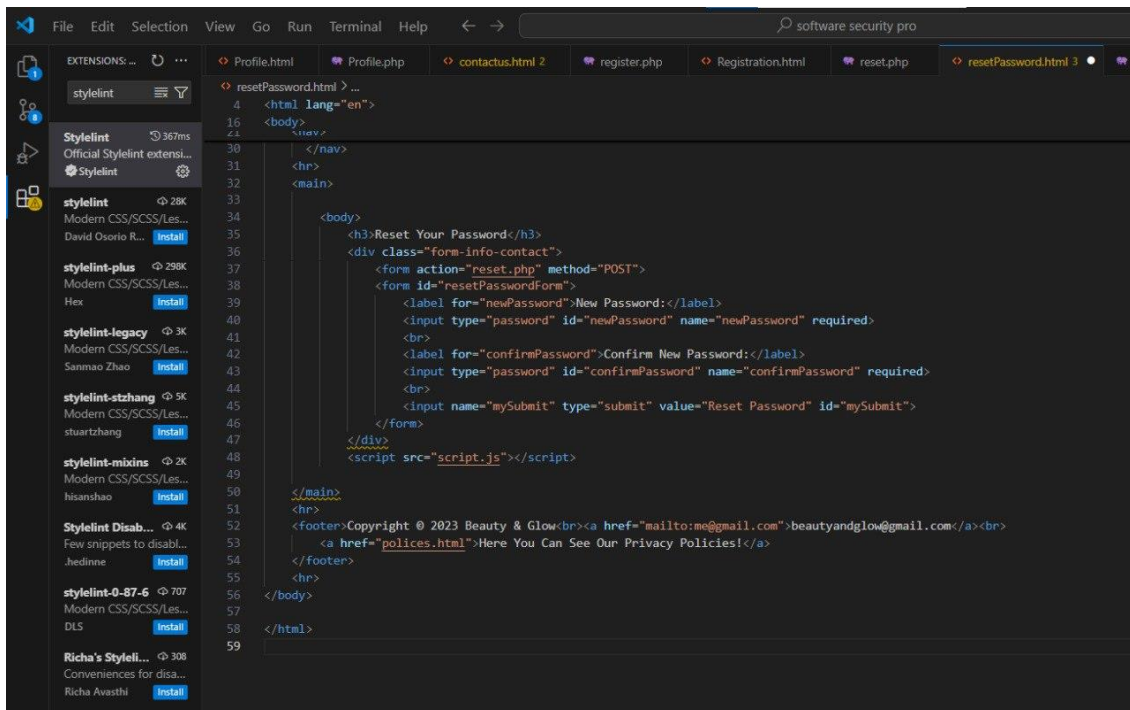
- For HTML:



The screenshot shows the Visual Studio Code editor with the file `resetPassword.html` open. The breadcrumb trail at the top of the editor reads: `resetPassword.html > html > body > main > body > div.form-info-contact > form > form#resetPasswordForm > label`. The code in the editor is as follows:

```
4 <html lang="en">
16 <body>
24 <nav>
30 </nav>
31 <hr>
32 <main>
33
34 <body>
35 <h3>Reset Your Password</h3>
36 <div class="form-info-contact">
37 <form action="reset.php" method="POST">
38 <form id="resetPasswordForm">
39 <label for="newPassword">New Password:</label>
40 <input type="password" id="newPassword" name="newPassword" required>
41 <br>
42 <label for="confirmPassword">Confirm New Password:</label>
43 <input type="password" id="confirmPassword" name="confirmPassword" required>
44 <br>
45 <input name="mySubmit" type="submit" value="Reset Password" id="mySubmit">
46 </form>
47 </div>
48 <script src="script.js"></script>
49
50 </main>
51 <hr>
52 <footer>Copyright © 2023 Beauty & Glow<br><a href="mailto:me@gmail.com">beautyandglow@gmail.com</a><br>
53 <a href="polices.html">Here You Can See Our Privacy Policies!</a>
54 </footer>
55 <hr>
56 </body>
57
58 </html>
59 </html>
```

Figure 2 HTML code



The screenshot shows the Visual Studio Code editor with the file `resetPassword.html` open. The breadcrumb trail at the top of the editor reads: `resetPassword.html > ...`. The code in the editor is as follows:

```
4 <html lang="en">
16 <body>
24 <nav>
30 </nav>
31 <hr>
32 <main>
33
34 <body>
35 <h3>Reset Your Password</h3>
36 <div class="form-info-contact">
37 <form action="reset.php" method="POST">
38 <form id="resetPasswordForm">
39 <label for="newPassword">New Password:</label>
40 <input type="password" id="newPassword" name="newPassword" required>
41 <br>
42 <label for="confirmPassword">Confirm New Password:</label>
43 <input type="password" id="confirmPassword" name="confirmPassword" required>
44 <br>
45 <input name="mySubmit" type="submit" value="Reset Password" id="mySubmit">
46 </form>
47 </div>
48 <script src="script.js"></script>
49
50 </main>
51 <hr>
52 <footer>Copyright © 2023 Beauty & Glow<br><a href="mailto:me@gmail.com">beautyandglow@gmail.com</a><br>
53 <a href="polices.html">Here You Can See Our Privacy Policies!</a>
54 </footer>
55 <hr>
56 </body>
57
58 </html>
59
```

Figure 3 HTML code

Our Experience with HTMLHINT

In our previous web development course, we used a tool familiar with HTMLHint that was designed to identify and rectify issues within HTML code. This experience helped us understand the importance of maintaining clean and valid markup.

Identifying Issues

Using HTMLHint, we were able to spot various problems in our HTML files. For instance, we discovered that one of our files contained two closing `</html>` tags. Recognizing this error was crucial, as having duplicate tags can lead to unexpected behavior in web browsers. We promptly removed the extra tag, and everything worked perfectly afterward.

- For CSS:
This tool has been downloaded and used.

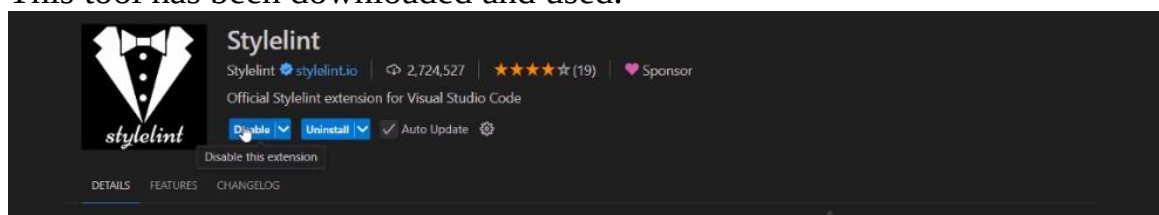


Figure 4 CSS Tool

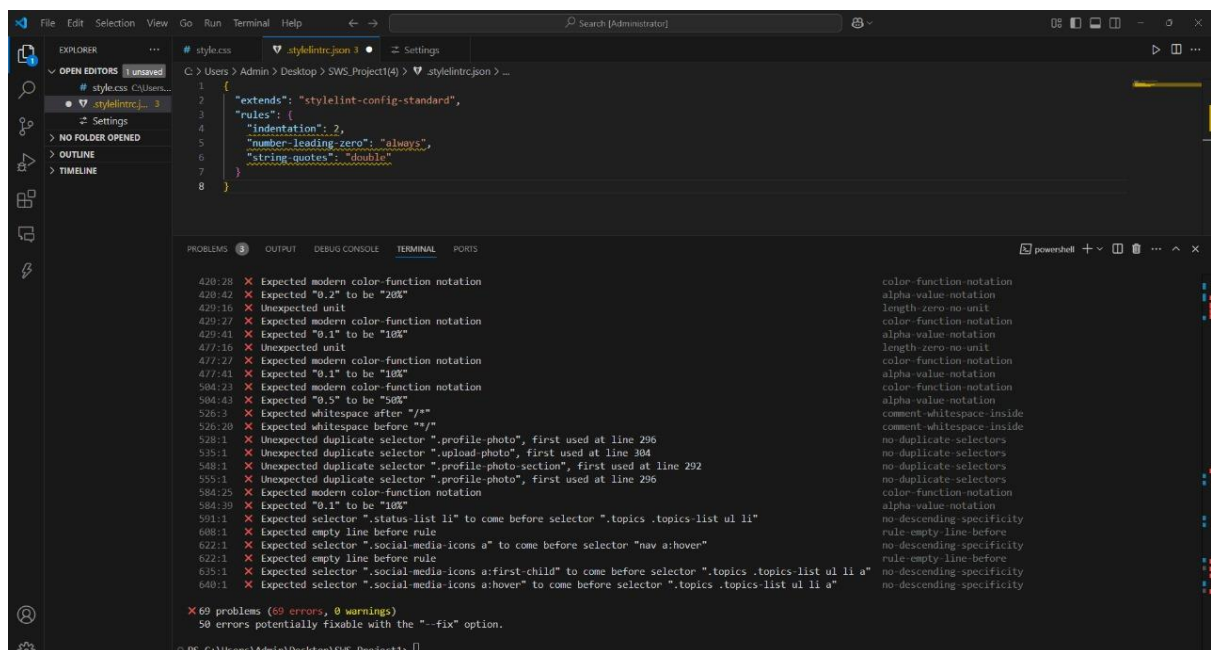


Figure 5 CSS Test

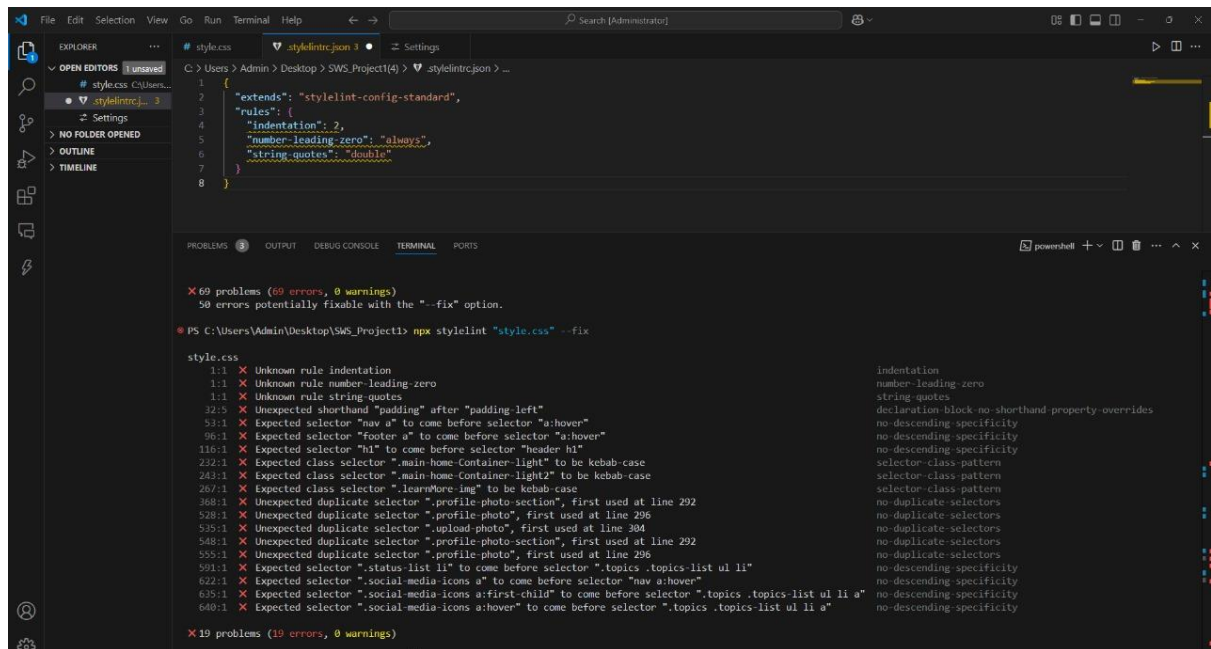


Figure 6 CSS Test

Our Experience with STYLELINT

we implemented stylelint in our project to help enforce consistent coding standards in our CSS. As we ran stylelint on our code, we encountered a significant number of warnings and errors, which prompted us to take a closer look at our coding practices.

Identifying Warnings and Errors

While analyzing our CSS, we noticed several common issues:

Selector Order: Many of our selectors were not in the expected order. For example, more specific selectors appeared before their less specific counterparts, which stylelint flagged as a problem.

Color Notation: We received warnings about our color values, particularly regarding the format we used. Stylelint encouraged us to adopt more modern practices, like using `rgba()` for colors with transparency.

Indentation and Consistency: Our code had inconsistent indentation and varied use of quotes for strings, making it harder to read and understand.

Fixing the Issues

To address these warnings, we collaborated as a team to make the necessary adjustments:

Reordered Selectors: We went through our CSS and ensured that selectors were arranged from the least specific to the most specific.

Updated Color Notations: We revised our color values to follow a more consistent format, switching to `rgba()` where appropriate.

Standardized Indentation: We agreed to adopt a two-space indentation throughout our styles, enhancing the overall readability of our code.

Establishing Rules

To prevent similar issues in the future, we decided to implement specific rules in our stylelint configuration. Here are the rules we established:

Indentation: We set the rule for two-space indentation to maintain consistency.

Leading Zero: We required leading zeros for decimal numbers to improve clarity.

String Quotes: We specified the use of single quotes for strings to ensure uniformity across our codebase.

by using stylelint, we identified and addressed numerous warnings in our CSS. The --fix option helped us clean up our code efficiently, improving its quality and consistency. This experience not only enhanced our coding practices but also helped us establish clear standards for future development.

- For PHP:

15	Cannot access property \$connect_error on mysqli false.	property.nonObject
16	Cannot access property \$connect_error on mysqli false.	property.nonObject
20	Offset 'REQUEST_METHOD' might not exist on array<mixed>. This error would be reported if the reportPossiblyNonexistentGeneralArrayOffset: true parameter was enabled in your configuration file .	phpstanPlayground.configParameter
23	Parameter #1 \$string of function trim expects string, mixed given.	argument.type
24	Parameter #1 \$string of function trim expects string, mixed given.	argument.type
27	Cannot call method prepare() on mysqli false.	method.nonObject
28	Cannot call method bind_param() on mysqli_stmt false.	method.nonObject
29	Cannot call method execute() on mysqli_stmt false.	method.nonObject
30	Cannot call method get_result() on mysqli_stmt false.	method.nonObject
32	Cannot access property \$num_rows on mysqli_result false.	property.nonObject
33	Cannot call method fetch_assoc() on mysqli_result false.	method.nonObject
36	Cannot access offset 'login_attempts' on array<string, float int string null> false null.	offsetAccess.nonOffsetAccessible
36	Cannot access offset 'login_attempts' on array<string, float int string null> false null. This error would be reported if the reportPossiblyNonexistentGeneralArrayOffset: true parameter was enabled in your configuration file .	phpstanPlayground.configParameter

Figure 7 PHP Test

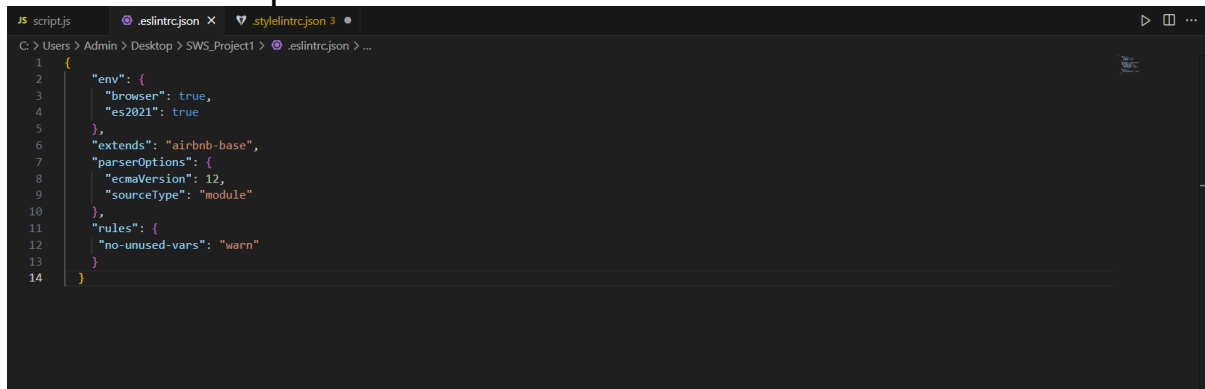
Our Experience with PHPStan

Errors While working on our PHP project, we decided to use PHPStan, a static analysis tool that helps us find potential issues in our code before it actually runs. As we ran PHPStan, we encountered several errors that were really helpful in pointing out areas we needed to fix. Here's a breakdown of some of the errors we faced:

1. Cannot access property `$connect_error` on `mysql|false`. What It Means: This error is telling us that we're trying to access the `$connect_error` property on a variable that could either be a `mysql` object or `false`. If the connection fails, the variable will be `false`, and we can't access properties on a boolean. How We Fixed It: We added a check to see if the variable is indeed a `mysql` object before accessing its properties. This way, we avoid any runtime errors.
2. Offset `'REQUEST_METHOD'` might not exist on `array<string, mixed>`. What It Means: This warning pops up when we try to access `$_SERVER['REQUEST_METHOD']`, but PHPStan is letting us know that this key might not exist in the array. How We Fixed It: We used `isset()` to check if the key exists before trying to access it. This makes our code safer.
3. Parameter #1 of `string()` expects `string`, given `mixed`. What It Means: This error indicates that we're passing a variable that PHPStan thinks isn't definitely a `string` to a function that requires a `string`. How We Fixed It: We made sure to validate our variables before passing them to functions, ensuring they were the correct type.
4. Cannot call method `bind_param()` on `mysql_stmt`. What It Means: This warning means we're trying to call the `bind_param()` method on something that PHPStan believes might not be a valid `mysql_stmt` object. How We Fixed It: We checked that the variable was properly initialized as a `mysql_stmt` before calling any methods on it.
5. Cannot call method `execute()` on `mysql_stmt`. What It Means: Similar to the previous error, this message tells us that we might be trying to call `execute()` on a variable that isn't recognized as a valid `mysql_stmt` object. How We Fixed It: We made sure to validate the variable before calling `execute()`.
6. Cannot call method `fetch_assoc()` on `mysql_result`. What It Means: This error shows up when we attempt to call `fetch_assoc()` on something that PHPStan doesn't recognize as a valid `mysql_result` object. How We Fixed It: Again, we checked that the variable was indeed a `mysql_result` before trying to call `fetch_assoc()`.
7. This error occurred because the `'reportPossiblyNonexistentGenericArrayOffset'` true parameter was

enabled in your settings. What It Means: This error lets us know that PHPStan is set to report potential issues when accessing array keys that might not exist, which helps us catch errors early. How We Fixed It: We reviewed how we were using array keys to ensure we were only accessing ones that actually existed. If needed, we could also change the PHPStan settings, but we decided to keep the checks for now. Using PHPStan was eye-opening!

- For JavaScript:



```
1 {
2   "env": {
3     "browser": true,
4     "es2021": true
5   },
6   "extends": "airbnb-base",
7   "parserOptions": {
8     "ecmaVersion": 12,
9     "sourceType": "module"
10  },
11  "rules": {
12    "no-unused-vars": "warn"
13  }
14 }
```

Figure 8 JavaScript code

Our Experience with ESLINT

Environment Setup: We specified that our code runs in a browser environment and supports ECMAScript 2021 features.

Style Guide: We chose to extend the Airbnb style guide, which is a popular choice for coding standards.

Parser Options: We configured the parser to understand modern JavaScript syntax and modules.

Rules: We set the no-unused-vars rule to warn us about unused variables or functions instead of throwing errors.

After setting up the configuration, we ran ESLint on our script.js file using the command:

```
npx eslint script.js
```

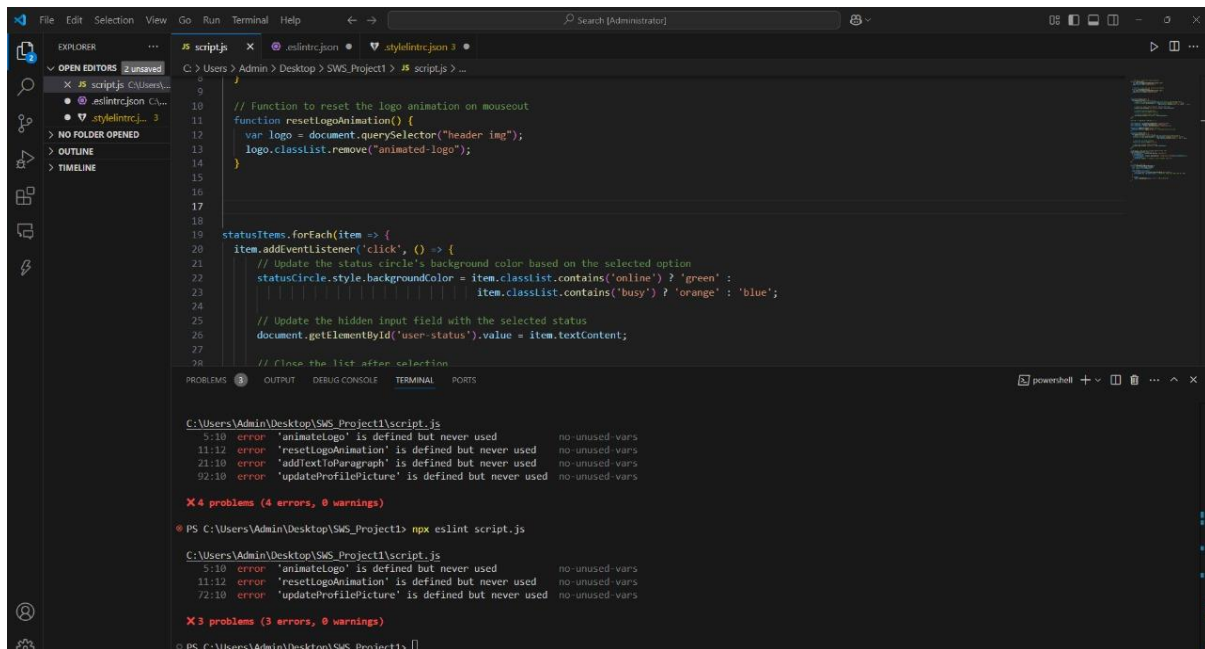


Figure 9 JavaScript Test

In our JavaScript code, we noticed that ESLint was warning us about some unused functions. Specifically, we found that the `readMore` function was not being used anywhere in our script, so we decided to remove it to clean up our code.

However, the other functions, like `animateLogo` and `resetLogoAnimation`, are indeed used in our code, even though ESLint flagged them as unused. This might be due to how ESLint interprets our code or the specific context in which those functions are called.

By addressing the warning for `readMore`, we've made our code cleaner, while we'll investigate why ESLint is flagging the other functions to ensure everything is working as expected.